

Rewriting Bun in Rust: Architectural Transformation, Safety, and Engineering Lessons

Author: Jarred Sumner | **Date:** July 8, 2026 | **Source:** Bun Official Engineering Blog

Executive Summary

Bun was originally created to deliver an all-in-one, ultra-fast JavaScript and TypeScript runtime, bundler, package manager, and test runner. Written in **Zig** during its initial single-developer phase, Zig provided the exact low-level control, zero-cost abstractions, and blazing execution speed required to launch the project rapidly.

However, as Bun grew to power over 22 million monthly downloads and became a primary runtime for industry leaders like Anthropic, Vercel, Railway, and DigitalOcean, maintaining memory safety in a massive codebase mixed with garbage-collected JavaScript objects became increasingly challenging.

In July 2026, the Bun core team announced a complete architectural rewrite of Bun's core engine from Zig to **Rust**. By leveraging advanced automated refactoring workflows using Anthropic's Claude models, the team successfully ported over 500,000 lines of Zig to Rust in an unprecedented 11-day sprint, eliminating vast categories of memory safety bugs while preserving Bun's industry-leading performance.

1. The Historical Context & The Zig Era

1.1 The Initial Vision and Scope

In April 2021, Bun started as a line-for-line port of `esbuild`'s JavaScript/TypeScript transpiler from Go to Zig. Over the course of a year, the scope expanded into a unified toolchain:

- **Transpiler & Bundler:** High-speed JS, TS, and CSS minification and bundling.
- **Package Manager:** `npm`-compatible client with global caching and fast lockfile resolution.
- **Test Runner:** Jest-compatible test runner with built-in mocking and snapshotting.
- **Node.js Compatibility Layer:** Re-implementations of native Node APIs (`fs` , `net` , `tls` , `http` , `zlib` , `child_process`).
- **Built-in Servers:** Native HTTP/1.1, HTTP/2, HTTP/3, and WebSocket clients and servers.

1.2 Why Zig Was Essential

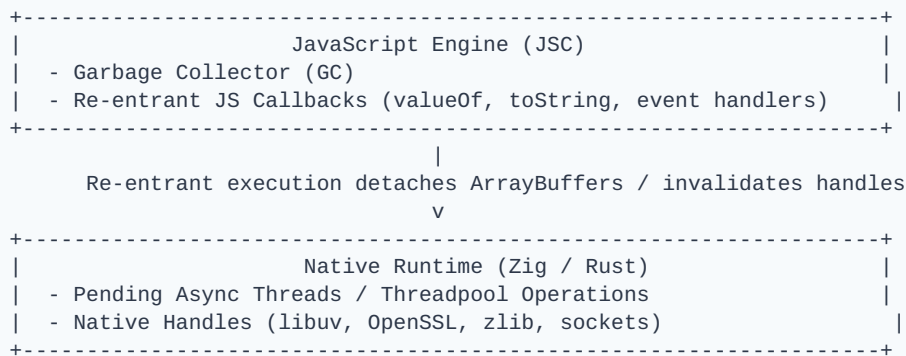
Without Zig, Bun would not have existed. Zig's explicit memory allocation (`std.mem.Allocator`), lack of hidden control flow, seamless C interoperability, and fast compilation enabled a single engineer to build a production-grade JavaScript runtime within 12 months.

2. The Memory Management Crisis

2.1 The Intersection of Garbage Collection and Native Memory

Bun embeds **JavaScriptCore (JSC)**, the C++ JavaScript engine powering Safari. JSC uses a conservative stack scanner and a garbage collector (GC). Conversely, native runtime memory in Zig (or C/C++) must be allocated and deallocated manually.

The core challenge arose from the interplay between JSC's GC cycles and native asynchronous IO callbacks. When JavaScript code executes callbacks (such as `valueOf()` , `toString()` , or event listeners) during active native IO operations, the JavaScript engine can mutate, detach, or collect underlying native buffers in the middle of native operations.



2.2 Representative Bug Analysis (Bun v1.3.14)

Prior to the Rust rewrite, the engineering team spent significant cycles fixing critical memory safety and concurrency bugs, including:

Category	Vulnerable Subsystem	Root Cause / Trigger Mechanism
Heap Use-After-Free	<code>node:zlib</code>	Calling <code>.reset()</code> on a stream while an async <code>.write()</code> was active on the threadpool.
Use-After-Free	<code>node:http2</code>	Re-entrant JS callbacks inside timeout listeners triggering hashmap re-hashes and invalidating internal stream pointers.
Buffer Detachment	<code>UDPSocket.send()</code>	User code in <code>valueOf()</code> callbacks detaching an <code>ArrayBuffer</code> between payload capture and native socket write.
Memory Leak	<code>crypto.scrypt</code>	Unreleased password/salt buffers when output buffer allocations failed during execution.
Double Free	CSS Parser	Double deallocation when parsing vendor-prefixed <code>background-clip</code> layered properties.
Race Condition	<code>MessageEvent</code>	GC marker thread reading torn variants in <code>m_data</code> during concurrent <code>BroadcastChannel</code> access.

2.3 The Limits of Testing and Tooling

Despite rigorous safety measures—including patching the Zig compiler for AddressSanitizer (ASAN) support, continuous 24/7 fuzzing via Fuzzilli, and running thousands of end-to-end memory leak tests—runtime detectors could only catch bugs that were explicitly triggered during execution. They could not provide compile-time guarantees.

3. The Structural Decision: Why Rust?

3.1 Evaluating Alternatives

The team evaluated several potential paths forward to solve stability at the root level:

- Strict Style Guides in Zig:** Requiring custom reference counting or smart pointers (`SharedPtr(T)`). This approach added verbose boilerplate and lacked compiler enforcement.
- C++ Migration:** C++ provided RAII and constructors/destructors, but still suffered from manual lifetime tracking, potential use-after-free errors, and heavy header overhead.
- Rust Rewrite:** Rust provided strict lifetime tracking, ownership rules, compile-time borrow checking, and automatic deterministic cleanup via the `Drop` trait.

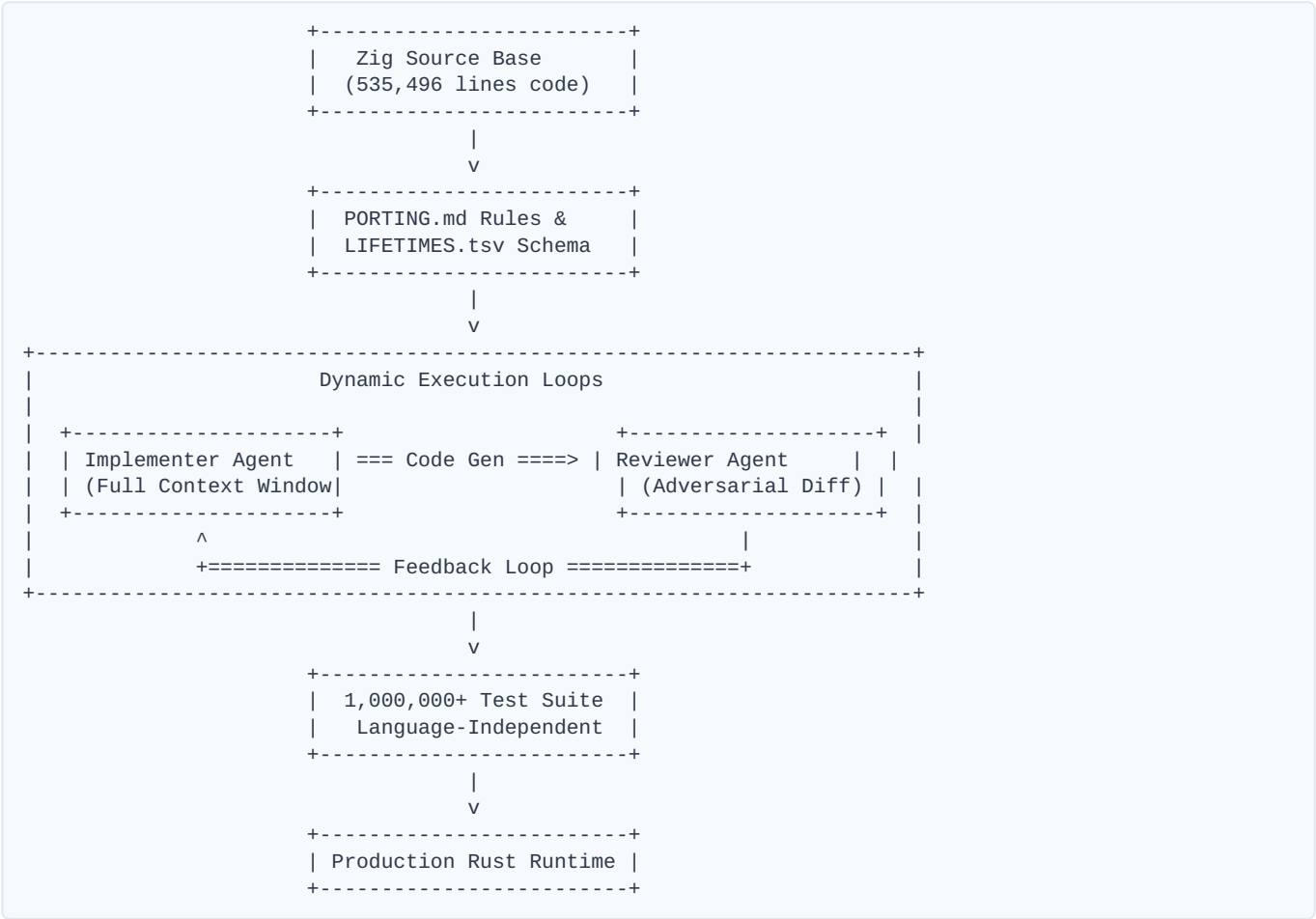
3.2 Comparison of Resource Management Approaches

Language	Cleanup Mechanism	Control Flow	Ownership Enforcement
Zig	<code>defer</code> / <code>errdefer</code>	Explicit at call site	Manual / Style Guide
C++	RAII (<code>~Destructor</code>)	Implicit on scope exit	Manual / Smart Pointers
Rust	<code>Drop</code> Trait	Implicit & Deterministic	Compile-Time Borrow Checker

By moving to Rust, the compiler transforms critical runtime vulnerabilities—such as use-after-free, double-free, and unhandled error paths—into immediate **compile-time errors**.

4. The AI-Driven Architecture & Migration Methodology

Rewriting 535,496 lines of complex Zig code traditionally takes a full engineering team over a year, requiring feature freezes and prolonged divergence. To prevent this, Bun utilized an automated, LLM-driven migration methodology using Anthropic's Claude models over an intensive **11-day process**.



4.1 Systemic Architectural Strategy

- **Mechanical Port First:** Rather than redesigning every abstraction immediately, the codebase was mechanically ported from Zig patterns to Rust equivalents to preserve identical performance characteristics and internal layouts.
- **Language-Independent Testing:** Because Bun's vast test suite (over 1,000,000 assertions) is written in TypeScript, the test runner validated runtime behavior independently of the underlying implementation language.

- **Adversarial Multi-Agent Review:**

- **Implementer Agents:** Given full context (original Zig code, porting rules, module specifications) to generate Rust code.
- **Adversarial Reviewer Agents:** Operating in isolated context windows with instructions to assume the code was broken, reviewing diffs strictly to catch edge-case bugs, memory leaks, and concurrency flaws before compilation.

5. Architectural Outcomes and Future Outlook

5.1 Technical Metrics & Gains

- **Safety Verification:** Entire categories of use-after-free and double-free bugs eliminated at compile time.
- **Maintainability:** Standardized error handling through Rust's `Result<T, E>` and deterministic resource cleanup via `Drop`.
- **C++ Integration:** Simplified interaction with C++ libraries (JavaScriptCore, BoringSSL, SQLite, uWebSockets) using Rust's rich FFIs and wrapper crates.

5.2 Key Engineering Takeaways

1. **Choose the Right Tool for the Phase:** Zig was the ideal choice for rapidly bootstrapping Bun from 0 to 1. Rust provided the long-term safety infrastructure required for a global runtime at enterprise scale.
2. **Decouple Test Suites from Implementation:** Having a language-agnostic test harness written in TypeScript was the single critical prerequisite that allowed a zero-regression language rewrite.
3. **Leverage Formal Verification and AI Loops:** Multi-agent adversarial feedback loops represent a fundamental shift in software engineering, enabling massive structural migrations without sacrificing code quality or stability.